PL/ SQL- CYCLE 2

EXP 1: CURSORS

AIM: To Practice and Implement Cursors

1. Create a PL/SQL block that declares a cursor. Display the empid ,name, designation and salary of employees who joined in a particular year. //CALL emp_joined_year(2003);

```
DELIMITER $$
CREATE PROCEDURE emp_joined_year(IN yr INT)
BEGIN
  DECLARE done INT DEFAULT FALSE;
  DECLARE v_empid INT;
  DECLARE v_name VARCHAR(100);
  DECLARE v_desig VARCHAR(50);
  DECLARE v_salary DECIMAL(10,2);
  DECLARE cur CURSOR FOR
    SELECT empid, CONCAT(fname,' ',lname), designation, salary
    FROM Employee
    WHERE YEAR(join_date) = yr;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
  OPEN cur;
  read_loop: LOOP
    FETCH cur INTO v_empid, v_name, v_desig, v_salary;
    IF done THEN
      LEAVE read_loop;
    END IF;
    SELECT v_empid, v_name, v_desig, v_salary;
  END LOOP;
  CLOSE cur;
END $$
DELIMITER ;
```

2. Create a PL/SQL block to display the empid , fname ,deptid of employees who work in a particular department.

```
DELIMITER $$
CREATE PROCEDURE emp_by_dept(IN d_id INT)
BEGIN
  DECLARE done INT DEFAULT FALSE;
  DECLARE v_empid INT;
  DECLARE v_fname VARCHAR(50);
  DECLARE v_deptid INT;
  DECLARE cur CURSOR FOR
    SELECT empid, fname, deptid
    FROM Employee
    WHERE deptid = d_id;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
  OPEN cur;
  read_loop: LOOP
    FETCH cur INTO v_empid, v_fname, v_deptid;
    IF done THEN
      LEAVE read_loop;
    END IF;
    SELECT v_empid, v_fname, v_deptid;
  END LOOP;
  CLOSE cur;
END $$
DELIMITER ;
```

3. Create a PL/SQL block to compute the average salary of employees and display the name and salary which are above and below average salary.

```
DELIMITER $$
CREATE PROCEDURE avg_salary_check()
BEGIN
  DECLARE done INT DEFAULT FALSE;
  DECLARE v_empid INT;
```

```
    DECLARE v_name VARCHAR(100);
    DECLARE v_salary DECIMAL(10,2);
    DECLARE avg_sal DECIMAL(10,2);
    SELECT AVG(salary) INTO avg_sal FROM Employee;
    DECLARE cur CURSOR FOR
      SELECT empid, CONCAT(fname,' ',lname), salary FROM Employee;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
    OPEN cur;
    read_loop: LOOP
      FETCH cur INTO v_empid, v_name, v_salary;
      IF done THEN
        LEAVE read_loop;
      END IF;
      IF v_salary > avg_sal THEN
        SELECT v_empid, v_name, v_salary, 'Above Average';
      ELSE
        SELECT v_empid, v_name, v_salary, 'Below Average';
      END IF;
    END LOOP;
    CLOSE cur;
  END $$
  DELIMITER ;
```

4. List the names of employees who work in the same department in which John works.

```
SELECT fname, lname
FROM Employee
WHERE deptid = (
  SELECT deptid FROM Employee WHERE fname = 'John'
);
```

5. Create a PL/SQL block to increase the salary of the employee in Department 10. The salary increases by 5% for employees whose salary is less than 20000 & 10% for the employees whose salary is greater than 20000.

```
UPDATE Employee

SET salary =

  CASE

    WHEN salary < 20000 THEN salary * 1.05

    WHEN salary > 20000 THEN salary * 1.10

    ELSE salary

  END

WHERE deptid = 10;
```

EXP: 2 PROCEDURES AND FUNCTIONS

AIM: To Implement Procedures and Functions

Write PL/SQL procedure for the following:

1. Find the details of employees who joined in the year 2003. //CALL emp_joined_2003();

```
DELIMITER $$
CREATE PROCEDURE emp_joined_2003()
BEGIN
  SELECT *
  FROM Employee
  WHERE YEAR(join_date) = 2003;
END $$
DELIMITER ;
```

2. Find out details of employees whose lname matches with a specified name (name to be given as a parameter). //CALL emp_by_lname('Smith');

```
DELIMITER $$
CREATE PROCEDURE emp_by_lname(IN p_lname VARCHAR(50))
BEGIN
  SELECT *
  FROM Employee
```

```
    WHERE lname = p_lname;
END $$
DELIMITER ;
```

Write PL/SQL function for the following:

1. Find the total no of employees in a particular dept(deptid to be given as parameter).

```
//SELECT total_emp_dept(10);
DELIMITER $$
CREATE FUNCTION total_emp_dept(p_deptid INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE total INT;
    SELECT COUNT(*) INTO total
    FROM Employee
    WHERE deptid = p_deptid;
    RETURN total;
END $$
DELIMITER ;
```

2. Display the name of the department with lowest average basic pay.

```
SELECT d.deptname
FROM Department d
JOIN Employee e ON d.deptid = e.deptid
GROUP BY d.deptid, d.deptname
ORDER BY AVG(e.salary) ASC
LIMIT 1;
```

3. Update the salary by 1% of the employee who has the highest salary.// CALL increase_highest_salary();

```
DELIMITER $$
CREATE PROCEDURE increase_highest_salary()
BEGIN
    UPDATE Employee
```

```
    SET salary = salary * 1.01
    WHERE salary = (
        SELECT max_sal FROM (
            SELECT MAX(salary) AS max_sal FROM Employee
        ) AS temp
    );
END $$
DELIMITER ;
```

EXP: 3 TRIGGER

AIM: To Implement Trigger .

Write PL/SQL Trigger for the following:

1. Write a trigger to ensure that no employee who is joined before 1960 can be inserted in the database.

```
DELIMITER $$
CREATE TRIGGER check_join_year
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    IF YEAR(NEW.join_date) < 1960 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee joining year cannot be before 1960';
    END IF;
END $$
DELIMITER ;
```

2. Create a trigger which will work before deletion in the employee table and create a duplicate copy of the record in another table employee_backup.

```
CREATE TABLE employee_backup LIKE Employee;

DELIMITER $$
CREATE TRIGGER backup_before_delete
```

```
BEFORE DELETE ON Employee
FOR EACH ROW
BEGIN
   INSERT INTO employee_backup
   VALUES (OLD.empid, OLD.fname, OLD.lname, OLD.designation,
         OLD.salary, OLD.deptid, OLD.join_date);
END $$
DELIMITER ;
```

4. Write a trigger to count the number of new tuples inserted using each insert statement.

```
CREATE TABLE insert_count (
   total_insert INT
);
INSERT INTO insert_count VALUES (0);

DELIMITER $$
CREATE TRIGGER count_insert
AFTER INSERT ON Employee
FOR EACH ROW
BEGIN
   UPDATE insert_count
   SET total_insert = total_insert + 1;
END $$
DELIMITER ;

SELECT * FROM insert_count;
```